



GROUP ASSIGNMENT

TECHNOLOGY PARK MALAYSIA

CT074-3-2-CC

CONCURRENT PROGRAMMING

APU2F2509CS(AI)

INDIVIDUAL ASSIGNMENT REPORT

HAND OUT DATE : 16th March 2026

HAND IN DATE : 24th May 2026

WEIGHTAGE : 20%

Student Name	TP Number
Ng Wei Yang	TP074904

Table of Contents

Table of Contents	i
1 Introduction and Background	1
1.1 The Scenario	1
1.2 System Assumptions	1
2 Explanation of Safety Aspects	3
2.1 Mutual Exclusion	3
2.2 Deadlock Prevention	3
3 Justification of Coding Techniques	4
3.1 Compliance with Restrictions	4
3.2 The Monitor Design	4
4 System Implementation and Testing	5
4.1 Basic Requirements	5
4.1.1 Mutual Exclusion on a Single Runway	5
4.1.2 Maximum Capacity of 3 Airplanes	6
4.1.3 Parallel Execution of Ground Operations	7
4.2 Additional Requirements	8
4.2.1 Single Mutually Exclusive Refuelling Truck	8
4.2.2 Congested Scenario with Priority Emergency Yeilding	9
4.3 Requirements NOT met	9
5 Depth of Discussion	10
5.1 Summary of Concurrency Concepts	10
5.2 Challenges and Solutions	10
6 UML	11
7 References	12

1 Introduction and Background

1.1 The Scenario

This case study aims to simulate the simultaneous operations of an airport called Asia Pacific Airport with constraints to mimic real airport conditions using concurrent programming techniques in Java. The objective is to manage six different airplanes arriving at random intervals of 0, 1 or 2 seconds, manage traffic and prevent collisions and enforce resource limitations. For example, only one airplane can use the runway at a time since the case study specifies that it's a small airport with no waiting area on the ground. Additionally, there are only 3 gates so that means that at any given time there can only be 3 planes on airport grounds. Furthermore, there is only one refuelling truck that can only be used by one plane at a time. The system must be able to handle all these concurrent tasks safely without using automated concurrency libraries like `java.util.concurrent`.

1.2 System Assumptions

To mimic realistic simulations as closely as possible while adhering to the constraints, several system assumptions were made:

- **Passenger Capacity:** It is assumed that each airplane can accommodate up to a maximum of 50 passengers. To simulate this, a random number generator is used to get the number of passengers for each plane when the threads are created.
- **Terminal Efficiency:** It is assumed the passenger terminal always has the capacity to handle embarking and disembarking crowds efficiently and that no delays will occur from tardy passengers.

- **Time Scaling:** To simulate the physical time requirements of each airport logistics, events like passenger boarding and disembarkation, landing, refuelling and cleaning take set amounts of time. Since the requirements outlined that each step should take some time while strictly requiring the program to finish running within 60 seconds, a scaled-down time model was utilized. Time is scaled down to the point where major events take between 500ms to 4000ms using *Thread.sleep()* while micro-actions like putting trash into a bag are abstracted into their parent processes to ensure the simulation finishes in time and the terminal doesn't get flooded with micro-actions (Oracle, n.d.-d).
- **Concurrent Events:** Another assumption is that once an airplane is safely docked at a gate, refuelling of the plane, the cleaning crew and the passenger disembarking and boarding can all operate simultaneously along with the other docked airplanes as parallel or interleaved threads.
- **Gate Allocation:** It is also assumed that planes do not have pre-assigned gates before arriving and will be dynamically assigned to the first available empty gate by the ATC based on their arrival time in a First-In-First-Out (FIFO) queue.
- **Emergency Scenario:** To meet the additional requirements of a congested scenario where 2 planes have already been docked in 2 of the 3 gates at the airport and 2 planes are waiting to land and a 3rd plane comes in with a fuel shortage, requiring an emergency landing, it is assumed that one specific airplane (Plane 5) is hardcoded to declare an emergency upon arrival to reliably trigger the ATC radar, allowing it to bypass the normal queue while adhering to the random (0, 1, 2 seconds) arrival time.

2 Explanation of Safety Aspects

2.1 Mutual Exclusion

Mutual Exclusion was enforced on all shared resources like the Runway, Gates and Refuelling Truck to prevent data race condition and simulate safety in the airport. Since automated libraries are strictly prohibited, the synchronized keyword was used to implement intrinsic locks (Oracle, n.d.-c). An example of this is when an aircraft attempts to land or take off, the Runway object acts as a Monitor, encapsulating the runway usage in a synchronized block. This guarantees that only one thread can acquire the intrinsic lock at a time, imagine Plane-1 holds the lock, Plane-2 is blocked from entering the critical section, preventing thread interference and eliminating the possibility of collisions. Similar intrinsic locks are also used to protect the Linked List queue to ensure data is not corrupted by simultaneous thread modifications.

2.2 Deadlock Prevention

Deadlocks often occur when competing for a shared resource in concurrent systems and in this case, this would include the runway and refuel truck. To avoid having multiple planes waiting indefinitely for these shared resources, such as when waiting for a lock held by another thread without releasing their own, hence a Guarded Block pattern was implemented with wait() and notifyAll() methods (Oracle, n.d.-b) and a LinkedList to enforce a First-In-First-Out (FIFO) queue.

An example would be when a plane requests to land which requires the runway resource, and while it is still occupied by another plane, the requesting plane enters a while loop and invokes wait(), this is critical since this allows the occupying plane to finish its task with the runway

which then invoke `notifyAll()` which wakes up all waiting threads in the queue, giving them a chance to check their while loop for the proper conditions to be allowed to access the runway. This strategy ensures fairness, prevents deadlocks and avoids continuous polling from flooding the terminal with requests.

3 Justification of Coding Techniques

3.1 Compliance with Restrictions

While adhering to the constraints, the parallel operations of an airport were simulated by using native Java Thread class for planes and Runnable interface for ground operations like passenger onboarding and disembarking, cleaning, refuelling (Oracle, n.d.-a). This architecture choice allows a docked plane thread to natively start multiple Runnable tasks simultaneously to make the simulation more realistic.

3.2 The Monitor Design

To enforce mutual exclusion in the system and prevent race conditions, custom Monitor objects were created using synchronized keyword (Oracle, n.d.-c). For example, the runway uses a static class-level intrinsic lock (`public static final Object RUNWAY_LOCK`) and by wrapping critical sections of landing and take-off methods within a `synchronized(RUNWAY_LOCK)` block, it guarantees that only one thread can occupy the runway at a time. Any next thread that wants to enter this block will safely be suspended until the lock is released, simulating real-world safety.

4 System Implementation and Testing

4.1 Basic Requirements

4.1.1 Mutual Exclusion on a Single Runway

A custom Monitor was used to make sure only one airplane uses the single runway at a time. The RUNWAY_LOCK is a static class-level object that provides an intrinsic lock, paired with a synchronized block to stop two planes from accessing the runway code at the same time.

```
public class Runway {  
    // Shared static lock object to enforce mutual exclusion across all instances  
    public static final Object RUNWAY_LOCK = new Object();  
    private final Queue<String> takeoffQueue = new LinkedList<>();  
}
```

Figure 1: Runway Intrinsic Lock Declaration

```
public void land(String planeName) {  
    // Synchronized block to prevent concurrent runway access  
    synchronized(RUNWAY_LOCK) {  
        System.out.println(planeName + ": Landing.");  
        try {  
            Thread.sleep(2000);  
        } catch (InterruptedException e) {  
            System.out.println(planeName + "'s runway operation was interrupted.");  
        }  
        System.out.println(planeName + ": Landed.");  
  
        // Wakes up any threads holding for emergency landings  
        RUNWAY_LOCK.notifyAll();  
    }  
}
```

Figure 2: Synchronized Landing Critical Section

4.1.2 Maximum Capacity of 3 Airplanes

To enforce strict capacity limit of 3 planes at the airport, the ATC acts as a Monitor using Guarded Blocks. If the planes on ground counter reaches 3, any following planes will be put into a while loop where they calls wait() and are safely paused until another plane leaves and calls notifyAll().

```
public class ATC {
    private int planesOnGround = 0;
    private boolean emergencyIncoming = false;
    private boolean landingSequenceActive = false;

    // First-Come-First-Serve queue to ensure fair thread scheduling
    private final Queue<String> waitingQueue = new LinkedList<>();
    private final Gate[] gates;
```

Figure 3: ATC State Variables

```
// Normal planes wait for sufficient capacity, no emergency, free runway and their turn in the queue
while (emergencyIncoming || planesOnGround >= 3 || !waitingQueue.peek().equals(planeName) || landingSequenceActive) {
    String currentReason;
    if (planesOnGround >= 3) {
        currentReason = "Airport Full";
    } else if (emergencyIncoming) {
        currentReason = "Airspace reserved for emergency";
    } else if (landingSequenceActive) {
        currentReason = "Runway currently in use";
    } else {
        currentReason = "Another plane is ahead in the queue";
    }

    if (!lastDenyReason.equals(currentReason)) {
        System.out.println("ATC: Landing Permission Denied for " + planeName + ". Reason: " + currentReason + ".");
        lastDenyReason = currentReason;
    }
    try {
        wait();
    } catch (InterruptedException e) {
        System.out.println("Main simulation thread was interrupted.");
    }
}
waitingQueue.poll();
planesOnGround++;
landingSequenceActive = true;
System.out.println("ATC: Landing permission granted for " + planeName + ".");
```

Figure 4: Capacity Enforcement Logic

4.1.3 Parallel Execution of Ground Operations

Concurrent operations of ground tasks are implemented using the Runnable interface to run multiple sub-threads simultaneously, then the main plane uses `.join()` to pause itself until all of them are completed.

```
// Start the ground jobs at the exact same time
Thread passengerThread = new Thread(new PassengerTask(planeName, passengerCount));
Thread cleaningThread = new Thread(new CleaningTask(planeName));
Thread refuelingThread = new Thread(new RefuelingTask(planeName, truck));
```

Figure 5: Thread Initialization

```
passengerThread.start();
cleaningThread.start();
refuelingThread.start();

try {
    // Thread Synchronization: Main Plane thread waits until sub-threads (ground tasks) complete
    passengerThread.join();
    cleaningThread.join();
    refuelingThread.join();
} catch (InterruptedException e) {
    System.out.println(planeName + " was interrupted during ground tasks.");
}
```

Figure 6: Thread Execution and Join Synchronization

4.2 Additional Requirements

4.2.1 Single Mutually Exclusive Refuelling Truck

Like the runway, the refuelling truck uses a custom Monitor to ensure thread safety. It uses an object-level lock `synchronized(this)` and a `LinkedList` to create a fair First-In-First-Out queue for the planes requesting to refuel.

```
public class RefuelingTruck {  
  
    private boolean inUse = false;  
    private final Queue<String> truckQueue = new LinkedList<>();  
}
```

Figure 7: Refuelling Truck State & Queue

```
public void refuel(String planeName) {  
    // Checking and acquiring the resource lock  
    synchronized(this) {  
        truckQueue.offer(planeName);  
  
        if (inUse || !truckQueue.peek().equals(planeName)) {  
            System.out.println(planeName + ": Waiting for the refueling truck. Truck is busy or another plane is ahead.");  
        }  
  
        // Thread waits until the truck is released & it is their turn in the queue  
        while (inUse || !truckQueue.peek().equals(planeName)) {  
            try {  
                wait();  
            } catch (InterruptedException e) {  
                System.out.println("Refueling truck was interrupted while queueing " + planeName + ".");  
            }  
        }  
        truckQueue.poll();  
        inUse = true;  
    }  
}
```

Figure 8: Fair Refuelling Monitor Logic

4.2.2 Congested Scenario with Priority Emergency Yielding

To simulate congested emergency scenario, a dynamic emergencyIncoming flag was added to the ATC which turns true when an emergency plane requests to land. Any normal plane that wakes up from notifyAll() is forced to yield their turn in the queue while the flag is true and go back to wait() so that the emergency plane can land first.

```
public class ATC {  
    private int planesOnGround = 0;  
    private boolean emergencyIncoming = false;  
    private boolean landingSequenceActive = false;
```

Figure 9: Emergency Flag Initialization

```
if (isEmergency) {  
    emergencyIncoming = true;  
  
    // Forces thread into WAITING state if the airport is full or the runway is busy  
    while (planesOnGround >= 3 || landingSequenceActive) {  
        String currentReason;  
        if (planesOnGround >= 3) {  
            currentReason = "Airport Full";  
        } else {  
            currentReason = "Runway currently in use";  
        }  
  
        if (!lastDenyReason.equals(currentReason)) {  
            System.out.println("ATC: Emergency Landing delayed for " + planeName + ". Reason: " + currentReason + ".");  
            lastDenyReason = currentReason;  
        }  
        try {  
            wait();  
        } catch (InterruptedException e) {  
            System.out.println("Main simulation thread was interrupted.");  
        }  
    }  
    planesOnGround++;  
    landingSequenceActive = true;  
    emergencyIncoming = false;  
    System.out.println("ATC: ***EMERGENCY Landing granted for " + planeName + ".*");  
}
```

Figure 10: Dynamic Emergency Priority Logic

4.3 Requirements NOT met

ALL Requirements were Fulfilled

5 Depth of Discussion

5.1 Summary of Concurrency Concepts

Overall, the airport simulation was successful while adhering to the constraints and requirements of the assignment. The parallel executions in the system are driven by the Thread class for independent planes, and their ground operations are encapsulated as Runnable tasks. To safely coordinate and manage these parallel processes Monitor objects paired with the synchronized keyword were applied to the ATC, Runway and Refuel Truck to enforce mutual exclusion and prevent race conditions. Furthermore, wait() and notifyAll() were implemented with a LinkedList FIFO queue to ensure fairness, enforce constraints and prevent deadlocks.

5.2 Challenges and Solutions

One challenge that I had to solve was inconsistencies between simulations when trying to create the additional feature of a congested emergency scenario where in my simulation Plane 5 declares a fuel shortage. Since the assignment requires random arrival time of 0, 1 or 2 seconds between thread instantiations, early aircraft arrival and completion of ground operations made the congested scenario inconsistent since they would leave before the emergency plane arrived. This was solved by extending the simulated time required for ground operations until it would always mathematically guarantee a congested scenario no matter the random factor of the arrival time.

Another hurdle was the handling of standard plane yielding its turn to a incoming emergency plane without using an early warning system to detect when the emergency plane spawns but only when receiving the request to land will the ATC be able to know it has to adjust and coordinate with accordingly. This was solved by integrating an emergencyIncoming flag to let

other planes requesting to land know to keep waiting while letting the emergency plane skip the queue and land.

6 UML

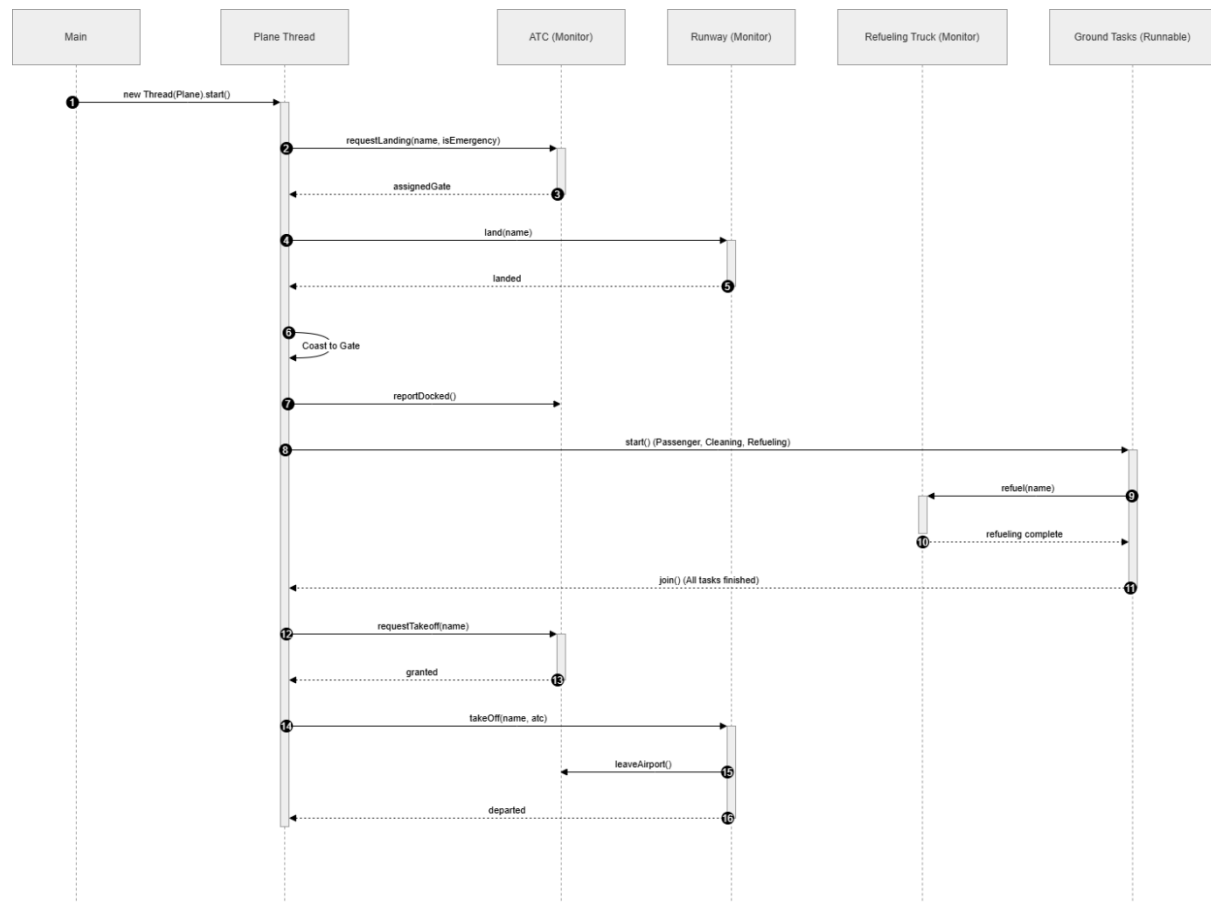


Figure 11: Sequence Diagram

(Word Count: 1451 words)

7 References

Oracle. (n.d.-a). Defining and Starting a Thread. The Java™ Tutorials.

<https://docs.oracle.com/javase/tutorial/essential/concurrency/runthread.html>

Oracle. (n.d.-b). Guarded Blocks. The Java™ Tutorials.

<https://docs.oracle.com/javase/tutorial/essential/concurrency/guardmeth.html>

Oracle. (n.d.-c). Intrinsic Locks and Synchronization. The Java™ Tutorials.

<https://docs.oracle.com/javase/tutorial/essential/concurrency/locksinc.html>

Oracle. (n.d.-d). Pausing Execution with Sleep. The Java™ Tutorials.

<https://docs.oracle.com/javase/tutorial/essential/concurrency/sleep.html>